Step 17 - GIC Interaction 1
Reference Note 1
Learning Objectives 1
1. What is the GIC? 1
2. Timer Expiry Does Not Directly Execute Software 1
3. High-Level Interrupt Flow 1
Complete Flow 1
4. GIC Interrupt States 1
Example 1
Inactive 1
Pending 1
Active 1
5. Timer Expiry to Pending State 1
6. Pending Does Not Mean Running 1
7. CPU Acceptance 1
Example Flow 1
8. ISR Entry 1
9. End Of Interrupt (EOI) 1
10. Why Timer Interrupts Can Reappear 1
Typical ISR Activity 1
11. Relationship Between Timer and GIC 1
Debugging Checklist 1
Architecture Summary 1
Common Misconceptions 1
Misconception 1 1
Misconception 2 1
Misconception 3 1
Misconception 4 1
Key Takeaways 1

Knowledge Check 1
Q1 1
Q2 1
Q3 1
Q4 1
Q5 1
Answer Key 1

Step 17 - GIC Interaction

Reference Note

This chapter is based on ARM Generic Timer and ARM Generic Interrupt Controller (GIC) architecture concepts.

In previous chapters we learned:

Counter
↓
Timer
↓
Timer Expiry

This chapter explains:

How does a timer expiry become CPU code execution?
What role does the GIC play?
When does an ISR actually start running?

A very important point:

Timer Expiry ≠ Interrupt Handler Execution

There are several steps in between.

Learning Objectives

After completing this chapter, you should understand:

- Purpose of the GIC
- Interrupt states
- Timer-to-GIC interaction
- PPI delivery
- Interrupt routing
- CPU interrupt handling
- End Of Interrupt (EOI)
- Why an expired timer does not immediately execute software

1. What is the GIC?

GIC stands for:

Generic Interrupt Controller

The GIC acts as the traffic controller for interrupts.

Without a GIC:

Timer
UART
Ethernet
DMA
GPIO

would all attempt to interrupt the CPU independently.

The GIC manages:

- Interrupt prioritization
- Interrupt masking
- Interrupt routing

- Interrupt acknowledgement
- Interrupt completion

2. Timer Expiry Does Not Directly Execute Software

Many engineers initially think:

Timer Expires
↓
ISR Executes

This is incorrect.

Actual flow:

Step	Action
1	Timer expires
2	PPI asserted
3	GIC marks interrupt pending
4	CPU accepts interrupt
5	Exception taken
6	ISR executes

3. High-Level Interrupt Flow

Component	Purpose
Timer	Detects expiry
PPI	Timer interrupt source
GIC	Interrupt manager
CPU Interface	Delivers interrupt
ISR	Software handler

Complete Flow

Step	Event
1	Counter reaches compare value
2	Timer expires
3	Timer asserts PPI
4	GIC receives interrupt
5	Interrupt becomes pending
6	CPU accepts interrupt
7	Exception handler entered
8	ISR executes
9	ISR acknowledges completion

4. GIC Interrupt States

Every interrupt exists in one of several states.

State	Meaning
Inactive	Not requested
Pending	Waiting for service
Active	ISR currently executing
Active + Pending	Re-triggered before completion

Example

Inactive

Timer not expired

No interrupt request.

Pending

Timer expired

Interrupt request exists.

CPU has not yet accepted it.

Active

CPU currently executing ISR

5. Timer Expiry to Pending State

Suppose:

Parameter	Value
Current Count	1,024,000
CVAL	1,024,000

Condition:

Current Count >= CVAL

Timer expires.

Timer output becomes asserted.

The GIC marks:

Interrupt = Pending

6. Pending Does Not Mean Running

A critical debugging concept.

Many conditions may prevent CPU delivery.

Examples:

Condition	Result
Interrupt masked	Not delivered
CPU IRQ disabled	Not delivered
Higher priority ISR active	Delayed
Interrupt not enabled in GIC	Not delivered

Thus:

Pending != ISR Running

7. CPU Acceptance

When the CPU is ready:

IRQ Enabled
Interrupt Higher Priority
Interrupt Enabled

then:

Pending → Active

The CPU vectors to the handler.

Example Flow

State	Description
Pending	Waiting
Active	ISR entered
Inactive	ISR completed

8. ISR Entry

Once accepted:

CPU automatically:

Action
Saves execution context
Updates exception state
Branches to interrupt vector
Begins ISR execution

At this point:

Software finally runs.

9. End Of Interrupt (EOI)

After processing:

ISR Complete

Software informs the GIC.

This is called:

EOI (End Of Interrupt)

Purpose:

Without EOI	With EOI
Interrupt remains active	Interrupt released

10. Why Timer Interrupts Can Reappear

Recall Step 15:

Current Count >= Compare Value

remains true after expiry.

Example:

Counter	Compare Value
1,024,000	1,024,000
1,024,100	1,024,000
1,024,200	1,024,000

Timer remains expired.

If software does not:

Program New Deadline

the interrupt may be asserted again.

Typical ISR Activity

Timer ISR usually:

Action
Read timer status
Program next expiry
Clear conditions
Return from interrupt

11. Relationship Between Timer and GIC

Timer Function	GIC Function
Detect expiry	Manage delivery
Assert interrupt signal	Track interrupt state
Generate event	Route event

Maintain timer status	Maintain interrupt status

Debugging Checklist

When:

Timer expired
But ISR never runs

check:

Item	Verify
ENABLE	Timer enabled
IMASK	Not masked
ISTATUS	Expired
PPI Enabled	Yes
GIC Enabled	Yes
IRQs Enabled	Yes

Architecture Summary

Component	Responsibility
Counter	Measures time
Timer	Detects expiry
PPI	Interrupt source
GIC	Interrupt management
CPU	Executes ISR
EOI	Completes interrupt processing

Common Misconceptions

Misconception 1

Timer expiry immediately executes ISR

False.

The interrupt must pass through the GIC.

Misconception 2

Pending means software is running

False.

Pending only means:

Interrupt waiting for service.

Misconception 3

EOI clears timer expiry

False.

EOI only updates interrupt state.

The timer condition must be handled separately.

Misconception 4

ISTATUS = 1 means ISR executed

False.

It only means:

Timer expired.

Key Takeaways

Concept	Key Point
Timer Expiry	Generates interrupt request
PPI	Timer interrupt source
GIC	Manages interrupt delivery
Pending	Waiting for CPU
Active	ISR executing
EOI	Completes interrupt handling
Expiry ≠ ISR	Several steps exist between them

Knowledge Check

Q1

Does timer expiry immediately execute software?

Q2

What component receives timer PPIs?

Q3

What does Pending mean?

Q4

What does Active mean?

Q5

What is EOI?

Answer Key

Question	Answer
Q1	No
Q2	GIC
Q3	Interrupt waiting for service
Q4	ISR currently executing
Q5	End Of Interrupt